SIMATS Engineering

ASSIGNMENT 3 – CO3 – Part 1 (Q1–Q4)

Course Code	DSA0104
Course Name	Object Oriented Programming with C++
Topic	Constructors, Destructors and Operator Overloading (CO3)
Nature of Assignment	Individual
Total Marks	50
Bloom's Level	K5 – Evaluate
Student Name	Sangeet T R
Register Number	1921260245
GitHub Repository	https://github.com/Sangeetthangaraj/DSA0104---object-oriented-programming-with-c-
Submission Date	05/11/26

Contents

Q1 Constructors and Overloading (6) · Q2 Copy Constructors (7) · Q3 Dynamic Constructors and Destructors (7) · Q4 Arithmetic and Stream Operators (7)

Build command used for every program: `g++ -std=c++17 -Wall -Wextra -g file.cpp -o out` (add `-fsanitize=address` where stated). Compiler: g++ ____ (fill in version, e.g. 13).

Note: Output blocks marked *Expected output* are what the code prints. Paste your own screenshot under each one after running, and fill every `_____` in the timing tables with your own measurements.

Q1 – Constructors and Constructor Overloading (6 Marks)

Design

A `Book` is created in three situations: full details, title only (donated) and empty record. One *target* constructor does all the work and validation; the other two delegate to it. Copies can never be negative: the target constructor throws `std::invalid_argument`, so an invalid Book is never constructed.

Implementation (`Task01/book.cpp`)

```
#include <iostream>
#include <stdexcept>
#include <string>

class Book {
    std::string title;
    std::string author;
    int copies;
public:
    // Target constructor: the ONLY place copies is checked
    Book(const std::string& t, const std::string& a, int c)
        : title(t), author(a), copies(c) {
        if (c < 0) throw std::invalid_argument("copies cannot be negative");
    }
    explicit Book(const std::string& t) : Book(t, "Unknown", 1) {}   //
donated
    Book() : Book("Untitled", "Unknown", 0) {}                       // empty
record
    friend std::ostream& operator<<(std::ostream& os, const Book& b) {
        return os << '[' << b.title << " | " << b.author << " | " << b.copies
<< ']';
    }
};

int main() {
    Book a("Wings of Fire", "A.P.J. Abdul Kalam", 5);
    Book b("Gitanjali");
    Book c;
    std::cout << a << '\n' << b << '\n' << c << '\n';
    try { Book bad("Bad", "X", -3); }
```

```
    catch (const std::invalid_argument& e) { std::cout << "Rejected: " <<
e.what() << '\n'; }
}
```

Expected output:

```
[Wings of Fire | A.P.J. Abdul Kalam | 5]
[Gitanjali | Unknown | 1]
[Untitled | Unknown | 0]
Rejected: copies cannot be negative
```

State of three objects after construction

Object	title	author	copies
a	Wings of Fire	A.P.J. Abdul Kalam	5
b	Gitanjali	Unknown	1
c	Untitled	Unknown	0

Design & Analysis

- **Default constructor supplied by the compiler:** only when the class declares *no* constructor at all. As soon as any constructor is written (e.g. the 3-argument one) the compiler stops supplying `Book()`. Here we wrote `Book()` ourselves.
- **Member initializer list vs assignment in the body:** the list *initializes* members directly. Assignment in the body first default-constructs the member and then assigns, which is a wasted step for `std::string`, and it is the only option for `const` members and references.
- **Order of initialization:** members are initialized in the order they are *declared* in the class (`title`, `author`, `copies`), never in the order written in the initializer list.
- **Delegation:** `Book(t)` and `Book()` call the target constructor, so the validation exists in one place only.

Experimental Investigation

Experiment	How	Result to record
Every constructor	program above	output shown
Remove default ctor, create `Book arr[3];`	delete `Book()`	error: `no matching function for call to 'Book::Book()'` (record your exact message)
Wrong order	declare `int copies; std::string title;` but write `: title(t), copies(c)`	warning: `-Wreorder` – member 'title' will be initialized after 'copies' (record exact text)
Negative copies	`Book("Bad","X",-3)`	exception caught, object not created

Design Question – `explicit`

Mark a single-argument constructor `explicit` when an *implicit* conversion would be surprising or dangerous. With `explicit Book(const std::string&)`:

```
Book b = std::string("Gitanjali");  // compiles WITHOUT explicit, ERROR with
explicit
Book b2(std::string("Gitanjali"));  // always fine (direct initialization)
```

Alternative Concept – one constructor with default arguments

```
Book(const std::string& t = "Untitled", const std::string& a = "Unknown", int
c = 0)
    : title(t), author(a), copies(c) { if (c < 0) throw
std::invalid_argument(".."); }
```

- **Time:** O(1) in both designs (one object, fixed number of field initializations). **Space:** O(1) in both (title and author are small strings).
- Default arguments give a single constructor and one validation point. The overloaded+delegation design also checks `copies` in one place (the target), but needs three declarations. The drawback of default arguments is that a donated book gets `copies = 0` instead of `1`, because the default is the same for every call; overloads let each situation choose its own defaults.

Q2 – Copy Constructors: Shallow and Deep Copies (7 Marks)

Version 1 – no copy constructor (the problem)

The compiler-generated copy constructor copies the pointer `medicines`, so `p` and `q` point to the *same* array (shallow copy). Changing `q` changes `p`, and when both destructors run, `delete[]` is called twice on the same block (double delete) and the program crashes.

```
Shallow copy                          Deep copy
p.medicines ──┐                       p.medicines ──► [Paracetamol][ORS]
              ├──► [Paracetamol][ORS]
q.medicines ──┘                       q.medicines ──► [Paracetamol][ORS]  (own
array)
```

Completed code (`Task02/prescription.cpp`)

```
#include <iostream>
#include <string>
class Prescription {
    int n;
    std::string* medicines; // dynamically allocated array
public:
    explicit Prescription(int count) : n(count), medicines(new
std::string[count]) {}

    // TODO 1: Copy constructor (deep copy)
    Prescription(const Prescription& o) : n(o.n), medicines(new
std::string[o.n]) {
```

```
        for (int i = 0; i < n; ++i) medicines[i] = o.medicines[i];
    }
    // TODO 2: Copy assignment
    Prescription& operator=(const Prescription& o) {
        if (this != &o) {                          // self-assignment guard
            std::string* t = new std::string[o.n]; // build new array first
            for (int i = 0; i < o.n; ++i) t[i] = o.medicines[i];
            delete[] medicines;                    // release old array
            medicines = t;
            n = o.n;
        }
        return *this;
    }
    ~Prescription() {
        // TODO 3
        delete[] medicines;
    }
    void set(int i, const std::string& m) { medicines[i] = m; }
    std::string get(int i) const { return medicines[i]; }
};
int main() {
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");
    Prescription q = p;      // copy constructor
    q.set(1, "Cetirizine");
    Prescription r(1);
    r = p;                   // copy assignment
    p.set(0, "Ibuprofen");
    std::cout << p.get(0) << ' ' << p.get(1) << " | " << q.get(0) << ' ' <<
q.get(1)
              << " | " << r.get(0) << ' ' << r.get(1) << '\n';
}
```

Expected output:

```
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS
```

Compile: `g++ -std=c++17 -Wall -Wextra -g -fsanitize=address prescription.cpp -o q2`

Sanitizer reports (record both)

- **Deep-copy version:** no errors (clean exit).
- **Version with the copy constructor deleted** (shallow): AddressSanitizer reports `attempting double-free` (or `heap-use-after-free`) at the second destructor. Paste your exact output here: _____

Design & Analysis

- **Why the shallow version crashes:** two objects own the same heap block; the first destructor frees it, the second frees it again (double delete) – undefined behaviour.
- **Rule of Three:** if a class needs a user-defined destructor, copy constructor *or* copy assignment, it almost certainly needs all three, because all three manage the same resource (the array): the

destructor releases it, the copy constructor and copy assignment must each give the new object its own copy.

- **Copy constructor is called when:** (1) an object is initialized from another (`Prescription q = p;`), (2) an object is passed by value to a function, (3) an object is returned by value (unless the copy is elided).

Experimental Investigation

Add `std::cout << "copy\n";` in the copy constructor.

Call style	Copies printed
`void f(Prescription x)` – by value	1
`void g(const Prescription& x)` – by const reference	0

Medicines n	Time to copy
10	____ µs
1,000	____ µs
100,000	____ µs

Measure with `std::chrono::high_resolution_clock` around `Prescription q = p;`. Expect time to grow roughly linearly with n.

Self-assignment test: `p = p;` – with the guard the object is unchanged; without it `delete[] medicines` would destroy the data before copying from it.

Conclusion

Rely on the compiler-generated copy constructor when the class holds only members that copy correctly by themselves (ints, `std::string`, `std::vector`, smart pointers with the intended semantics). Write your own when the class owns a raw resource (raw pointer, file handle, socket) that must be duplicated.

Alternative Concept – `std::vector<std::string>` (Rule of Zero)

```
class Prescription { std::vector<std::string> medicines; public: explicit Prescription(int n):medicines(n){} /* no copy ctor, no dtor */ };
```

Design	Copy time	Space
Raw array, deep copy	O(n)	O(n)
`std::vector`, compiler copy	O(n)	O(n)
Shallow copy (pointer only)	O(1)	O(1) – but unsafe

The vector version has the same cost as our deep copy, with no hand-written code and no double-delete risk.

Q3 – Dynamic Constructors and Destructors (7 Marks)

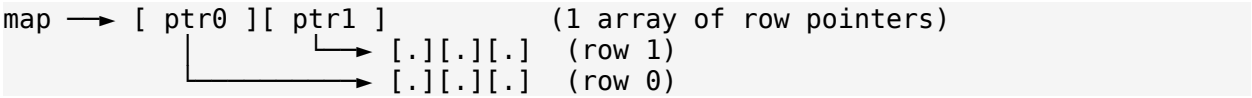
Implementation (`Task03/event.cpp`)

```
#include <iostream>
#include <string>
class Event {
    std::string name;
    int rows, seats, booked;
    char** map;   // rows pointers, each to an array of 'seats' chars
public:
    Event(const std::string& nm, int r, int s) : name(nm), rows(r), seats(s),
booked(0), map(new char*[r]) {
        for (int i = 0; i < rows; ++i) { map[i] = new char[seats]; for (int j
= 0; j < seats; ++j) map[i][j] = '.'; }
        std::cout << "[+] " << name << ' ' << rows << 'x' << seats << '\n';
    }
    Event(const Event&) = delete;             // owns raw memory: forbid
shallow copies
    Event& operator=(const Event&) = delete;
    bool book(int r, int s)   { if (r<0||r>=rows||s<0||s>=seats||map[r]
[s]=='X') return false; map[r][s]='X'; ++booked; return true; }
    bool cancel(int r, int s) { if (r<0||r>=rows||s<0||s>=seats||map[r][s]!
='X') return false; map[r][s]='.'; --booked; return true; }
    ~Event() {
        std::cout << "[-] " << name << ": " << booked << " of " << rows*seats
<< " booked\n";
        for (int i = 0; i < rows; ++i) delete[] map[i];
        delete[] map;
    }
};
int main() {
    Event a("Stack", 2, 3);
    { Event b("Block", 1, 2); }
    Event* h = new Event("Heap", 2, 2);
    a.book(0,1); a.book(1,2); a.cancel(0,1);
    h->book(0,0);
    delete h;
    std::cout << "main ends\n";
}
```

Expected output (order of construction/destruction):

```
[+] Stack 2x3
[+] Block 1x2
[-] Block: 0 of 2 booked
[+] Heap 2x2
[-] Heap: 1 of 4 booked
main ends
[-] Stack: 1 of 6 booked
```

Memory layout of the seat map (2 rows × 3 seats)



Design & Analysis

- **Order:** a local object is constructed where declared and destroyed when `main` ends, in reverse order of construction; a block-scope object is destroyed at the closing brace; a heap object is constructed by `new` and destroyed only by `delete`.
- **Forgotten `delete`:** destructor never runs, so the seat map leaks (ASan/LeakSanitizer reports a leak). **`delete` instead of `delete[]`:** undefined behaviour – for arrays the destructors/size bookkeeping are wrong (ASan: `alloc-dealloc-mismatch`).
- **Destructor takes no arguments and cannot be overloaded:** it is called automatically at end of lifetime, so there is no place to supply arguments, and there must be exactly one way to destroy an object.

Experimental Investigation

- Run with `-fsanitize=address`: (a) remove `delete h;` → record the leak report; (b) correct code → clean. Paste outputs: _____
- Zero rows: `Event z("Zero", 0, 5);` → `new char*[0]` is legal, the loop does nothing, no bookings possible, destructor prints `0 of 0 booked`.

Map size	Memory (rows×seats bytes + rows pointers)	Measured
10 × 20	200 B + 10×8 B = 280 B	____
100 × 200	20,000 B + 800 B = 20,800 B	____
1,000 × 2,000	2,000,000 B + 8,000 B = 2,008,000 B	____

Conclusion

With RAII the destructor releases memory automatically when the object goes out of scope (even when an exception is thrown), so memory cannot be forgotten or freed twice by the caller.

Predict the Output

My predicted output (written before running):

```
+A +B -B copyAc use -Ac +P -P end -A
```

Explanation: `a` is created (+A); `b` is created and destroyed at the end of its block (`+B -B`); `use(a)` takes the parameter *by value*, so the copy constructor runs and builds a temporary named `Ac` (`copyAc`), the function prints `use`, and the copy is destroyed when the call ends (`-Ac`); `p` is created and deleted (`+P -P`); `end` is printed; finally `a` is destroyed when `main` returns (`-A`). Compare with your actual run and explain any difference: _____

Alternative Concept – single block or `vector<vector<char>>`

Design	Access	Allocations	Extra space
Array of row pointers (`new` per row)	O(1) `map[r][s]`	rows + 1	rows pointers
One block, index `r*seats+s`	O(1)	1	none
`vector<vector<char>>`	O(1)	rows + 1	per-vector bookkeeping (~24 B per row)

The single block is the cheapest (one allocation, contiguous, cache friendly), and with `std::vector<char>(rows*seats)` the destructor can be dropped entirely.

Q4 – Overloading Arithmetic and Stream Operators (7 Marks)

Completed code (`Task04/money.cpp`)

```
#include <iomanip>
#include <iostream>
class Money {
    long long paise;
public:
    Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}

    // TODO 1: + as a member, returns a NEW Money
    Money operator+(const Money& o) const { Money r; r.paise = paise +
o.paise; return r; }

    // TODO 2: == as a member, << as a friend
    bool operator==(const Money& o) const { return paise == o.paise; }
    friend std::ostream& operator<<(std::ostream& os, const Money& m) {
        long long a = m.paise < 0 ? -m.paise : m.paise;
        if (m.paise < 0) os << '-';
        return os << "Rs " << a / 100 << '.' << std::setfill('0') <<
std::setw(2) << a % 100 << std::setfill(' ');
    }
};
int main() {
    Money a(120, 75), b(79, 25);
    Money c = a + b;
    std::cout << c << '\n' << std::boolalpha << (c == Money(200)) << '\n';
    std::cout << Money(0, 75) + Money(0, 50) << '\n';   // carry test
}
```

Expected output:

```
Rs 200.00
true
Rs 1.25
```

Full class for the report (extra operators)

Add to the class: `Money& operator+=(const Money& o){ paise += o.paise; return *this; }`, `Money operator-(const Money& o) const`, `bool operator<(const Money& o) const`, and `friend std::istream& operator>>(std::istream& is, Money& m)` (read rupees and paise, then set `m.paise`).

Design & Analysis

- **`+` returns a new object, `+=` returns `*this`:** `a + b` must not change `a` or `b`, so it builds a result. `a += b` modifies `a` and returns a reference so that chains like `(a += b) += c` work without a copy.
- **Members vs friends:** `+=`, `+`, `-`, `==`, `<` are members (they act on the left operand). `<<` and `>>` are non-members because the left operand is `std::ostream`/`std::istream`, which we cannot modify; they are friends only because they read the private `paise`.
- **Compiler translation:** `a + b` becomes `a.operator+(b)`; `std::cout << c` becomes `operator<<(std::cout, c)`.

Experimental Investigation

Test	Result
Rs 0.75 + Rs 0.50	125 paise → `Rs 1.25` (carry works because only paise are stored)
Subtraction giving negative	e.g. Rs 1.00 − Rs 2.50 = −150 paise → prints `-Rs 1.50`; decision: allow it (refunds/discounts), print the sign
1,000,000 additions of Rs 0.10	`Money`: exactly Rs 100000.00; `double` version: ____ (shows rounding error, e.g. 100000.00000133)
Readability	`total = price + tax - discount;` vs `total = subtract(add(price, tax), discount);` – operators read like arithmetic

Engineering Decision

Money × Money makes no sense (the result would be rupees²). Sensible operators: `+`, `-`, `+=`, `-=`, comparisons, `*` and `/` by a plain number (quantity, split bill), unary `-`. Not sensible: Money × Money, Money % Money.

Alternative Concept – non-member friends or named functions

```
friend Money operator+(const Money& a, const Money& b) { Money r; r.paise = a.paise + b.paise; return r; }
```

Time O(1) and space O(1) for all versions (member, friend, `add()`). The friend/non-member form allows `100 + m` as well as `m + 100`: the left operand can be implicitly converted via `Money(100)`. A member `operator+` only converts the *right* operand, so `100 + m` fails to compile.